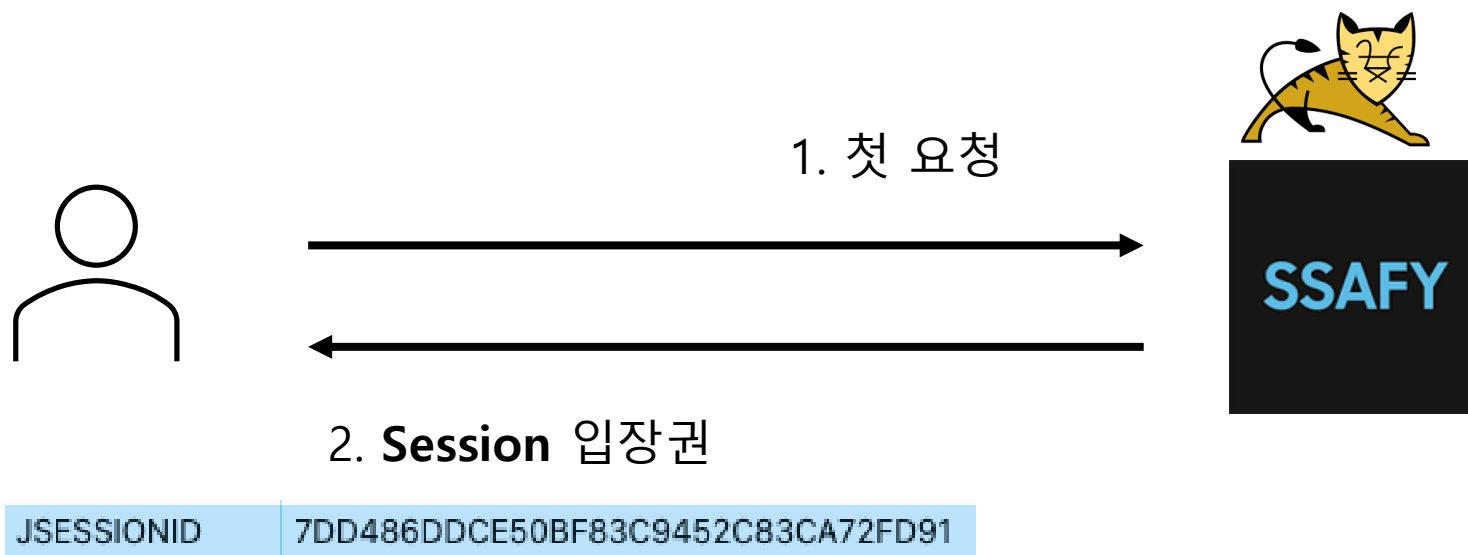

JWT는 언제 써야할까

CONTENTS

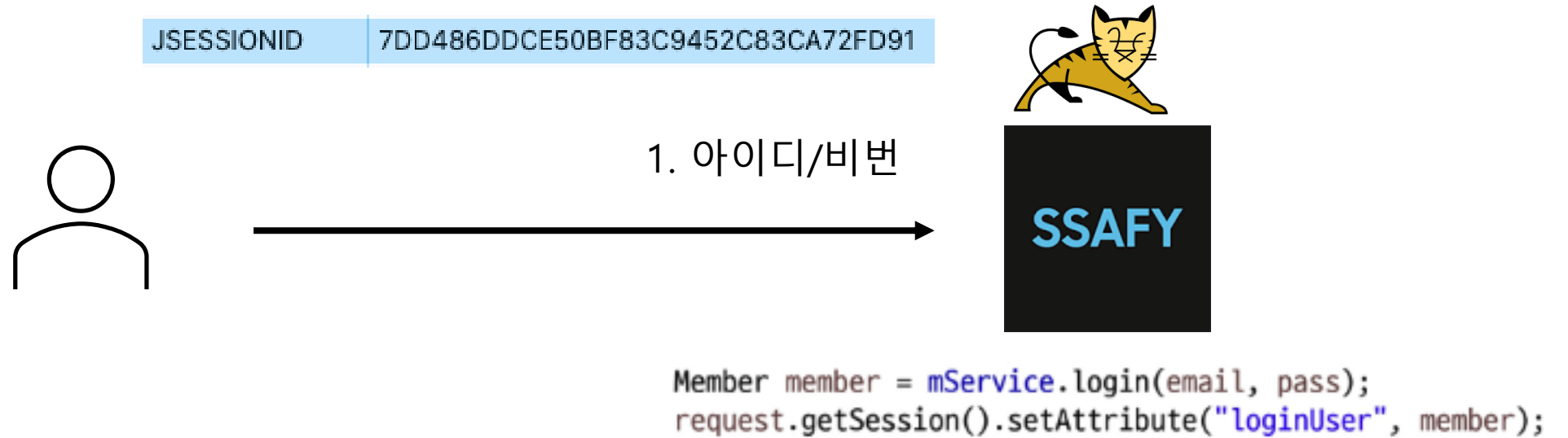
1

1. session
2. token(jwt)
3. session vs token
4. jwt 보안
5. session & JWT(AT, RT) 저장 위치
6. JWT는 언제 써야할까

1. session



1. session



1. session

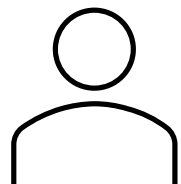
```
if(session.getAttribute("loginUser")!=null)
```

JSESSIONID

7DD486DDCE50BF83C9452C83CA72FD91

1. 강의 요청

2. 검증



3. 강의 제공

2. token(jwt)

<https://www.jwt.io>

JWS – 디지털 서명

JWE - 암호화

JWT Decoder JWT Encoder

Fill in the fields below to generate a signed JWT.

Generate example

HEADER: ALGORITHM & TOKEN TYPE

CLEAR

Valid header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

CLEAR

Valid payload

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

SIGN JWT: SECRET

CLEAR

Valid secret

a-string-secret-at-least-256-bits-long234234098

Encoding Format

base64url

UTF-8

• base64url

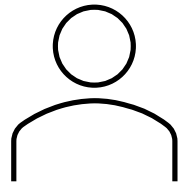
JSON WEB TOKEN

COPY

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6I6IkpvaG4gRG9lIiwiaWQiOiJhbnR5dWUzIiwiaWF0Ij0tYmN0.eyJYt8hKGKMKJVaeq46xhC086vAfJVA-ESN93gAK23Z0
```

2. token(jwt)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0nRydWU0ImVhdCI6MTUxNjIzOTAyMn0.dxndtMXHEydiuUA_c4HXNDFsbKK1v8GZgFJr7qy6IY
```



1. 강의 요청



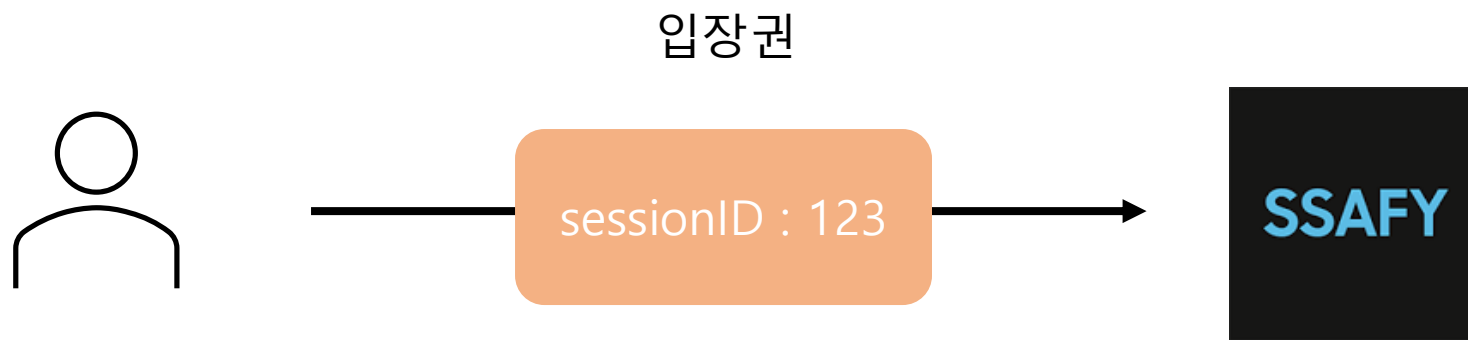
2. 검증



3. 강의 제공

3. session vs token

- sessionID 탈취 -> sessionID 가 123인 세션 초기화
- 서버에서 세션 상태를 관리해야하는 부담

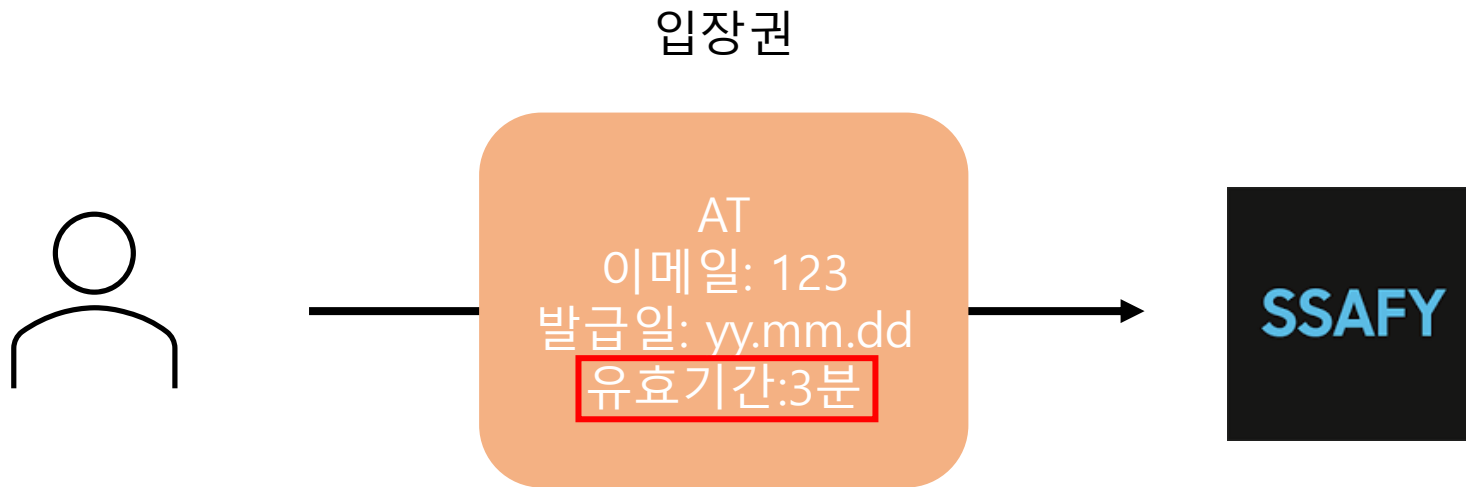


만료시간	sessionID
1시간	123
30분	456

3. session vs token

- 토큰에 회원정보, 만료기간 정보가 이미 들어있어서 따로 세션 객체를 관리할 필요가 없다.

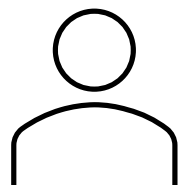
- 탈취 당하면 유효기간 까지 막을 수 없다.



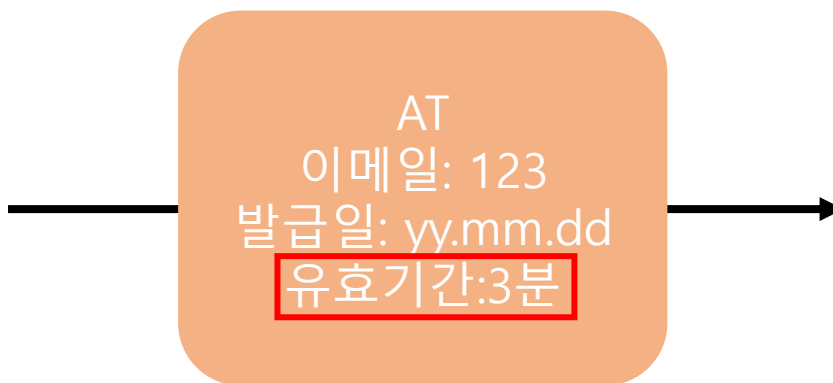
4. jwt 보안

4.1. jwt 블랙리스트

- 결국 session 방식과 동일



입장권



Jwt blacklist

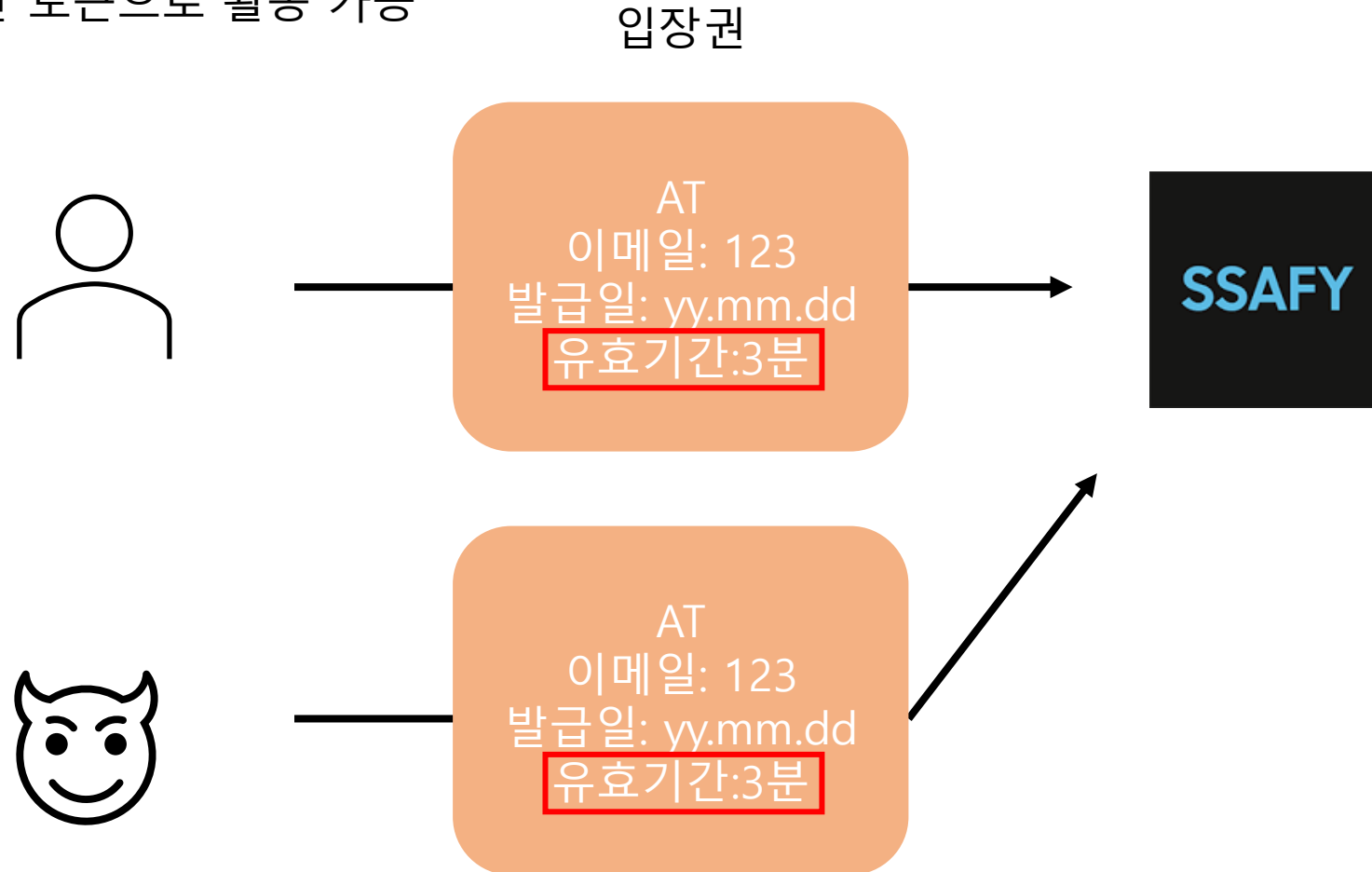
123

4. jwt 보안

4.2. jwt 유효기간 짧게

- 3분 동안만 탈취한 토큰으로 활동 가능

- 3분마다 로그인 직접 해야함.

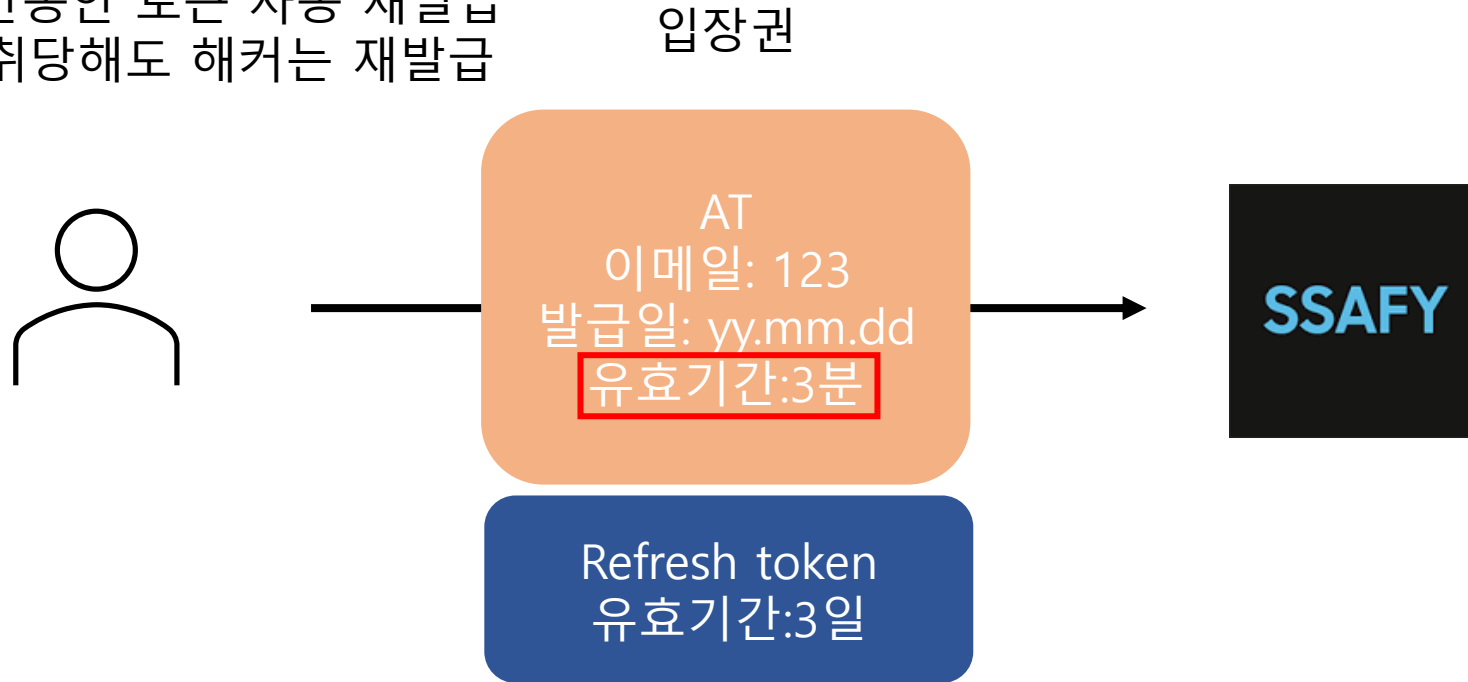


4. jwt 보안

4.2. jwt 유효기간 짧게

- Refresh token 기간동안 토큰 자동 재발급
- 주황색 토큰만 탈취당해도 해커는 재발급 불가능

- Refresh token까지 탈취당하면 해커도 재발급

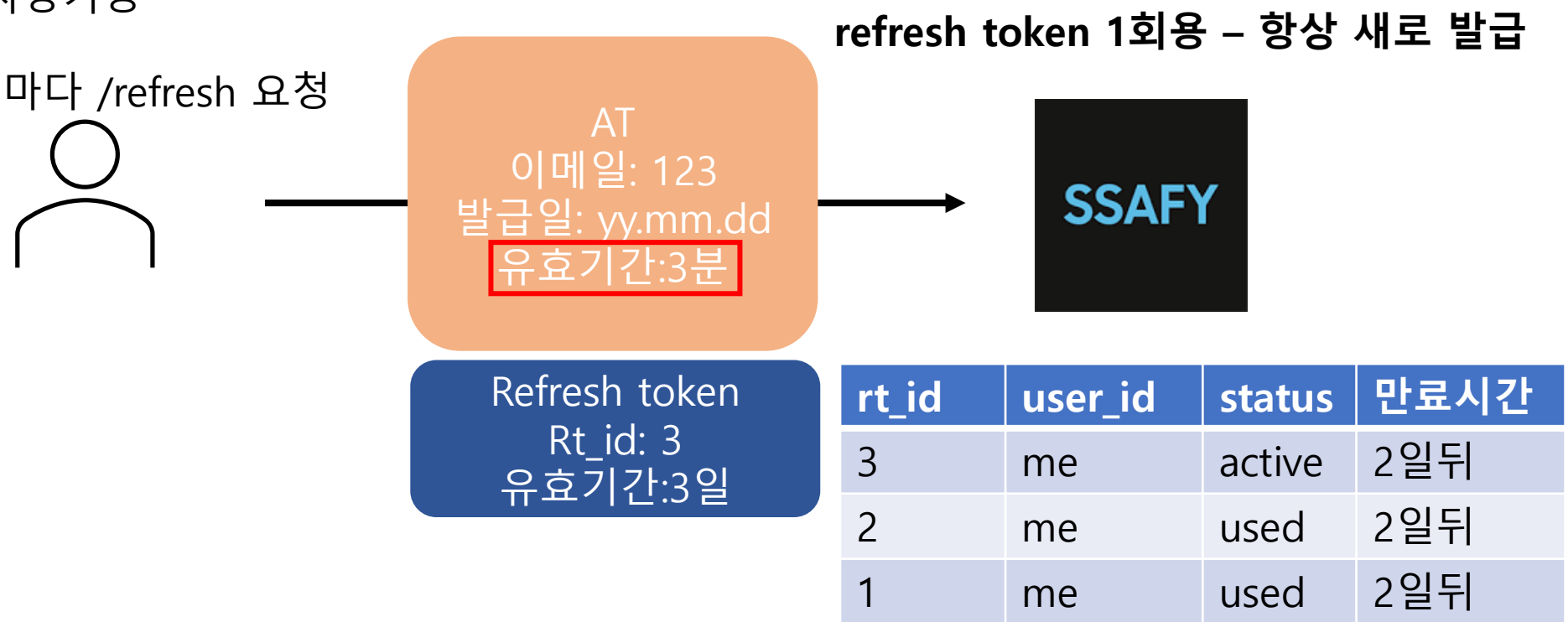


4. jwt 보안

4.3. refresh token rotation

- 가장 최근에 만들어져서 한번도 사용되지 않은 Refresh token만 사용가능
- 주황색 토큰 만료 마다 /refresh 요청

주황색 토큰 유효기간 동안은 **stateless** 하게 사용



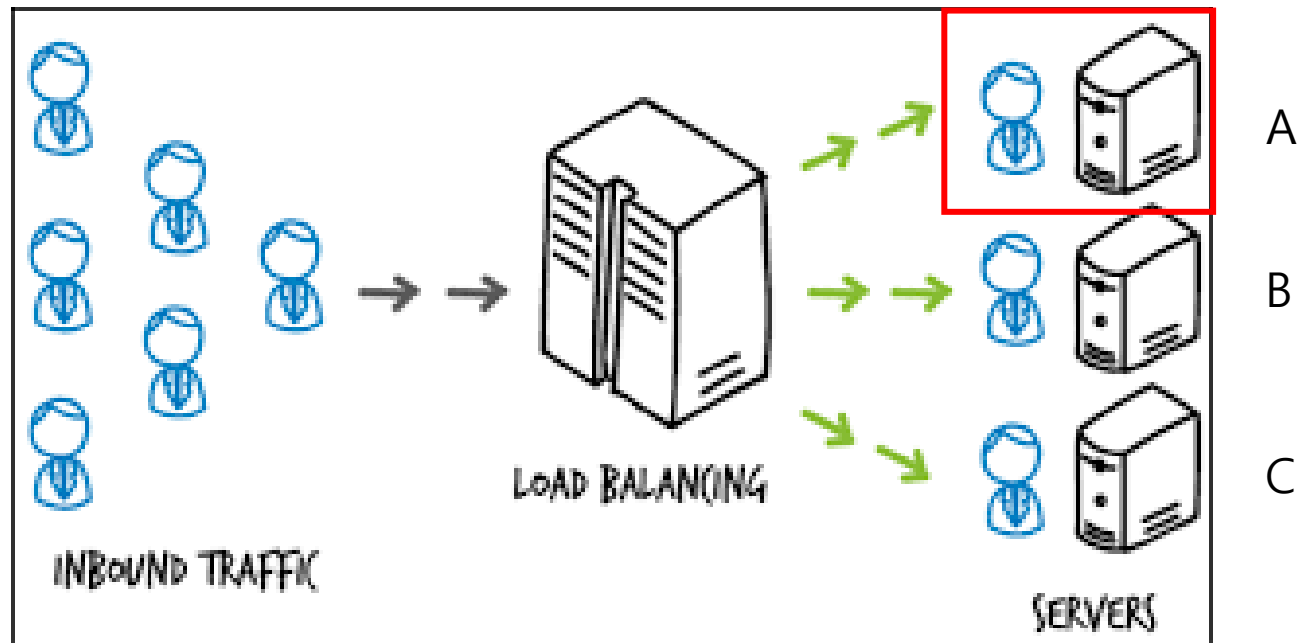
5. JWT는 언제 써야 좋을까

	session	JWT
요청 처리	모든 요청마다 세션ID 확인 → 서버 저장소 조회 필요	평소엔 토큰 검증만 → 서버 저장소 조회 불필요
로그아웃/강제 만료	서버가 세션 무효화 → 즉시 반영 가능	AT는 만료 전까지 유효, RT Rotation으로 제어
복잡도	구현 간단	설계 복잡, AT/RT 로직 + 보안 고려 필요
적합한 상황	사용자 수 적거나, 실시간 강제 로그아웃 중 요할 때	대규모 트래픽, 분산/마이크로서비스, 외부 API 연동이 많은 경우

6. session 저장 위치

6.1. WAS 인메모리 (톰캣, 서블릿 container)

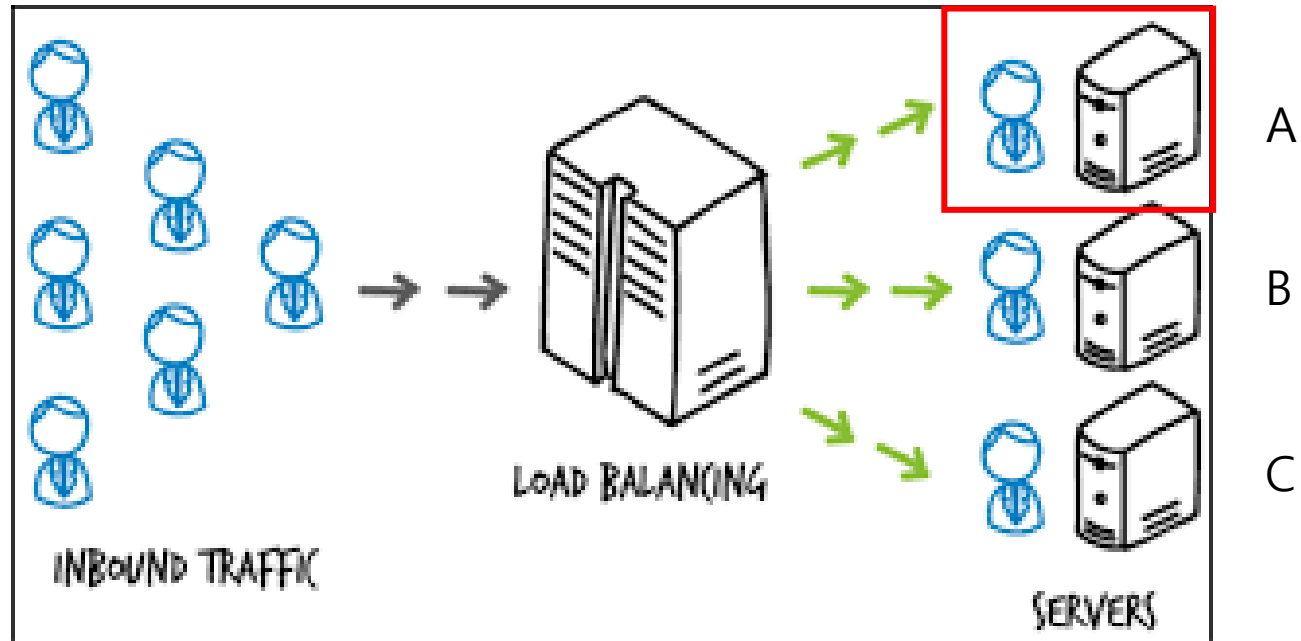
- WAS 서버가 JVM의 힙 메모리에 HttpSession 객체에 로그인 정보 저장
- 여러 개의 서버를 사용하는 경우 문제 발생



6. session & JWT(AT, RT) 저장 위치

6.2. Sticky session

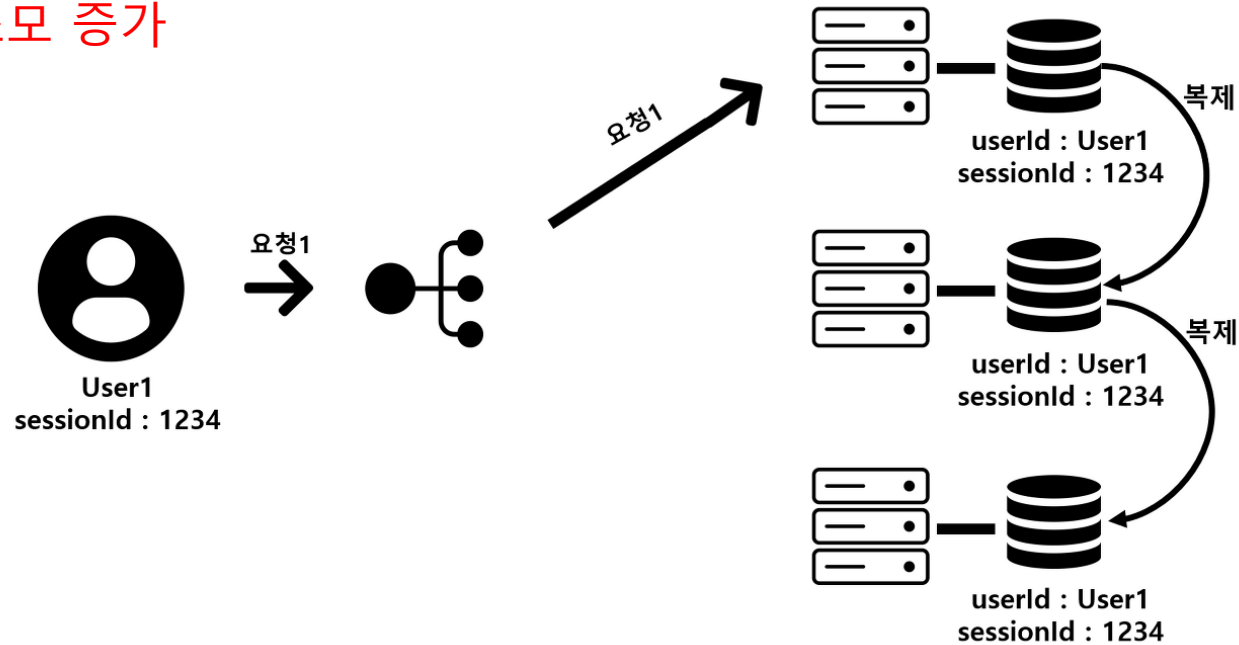
- 로드밸런서가 한번 로그인 했던 서버로 요청을 계속 보내도록 고정하는 방식
- 로드 밸런싱 분산 효과가 안좋아짐.
- 항상 보내던 서버가 다운 되면 다른 노드로 갔을 때 세션 정보가 없어짐



6. session & JWT(AT, RT) 저장 위치

6.3. 세션 복제(클러스터링)

- 다른 서버로 요청이 가도 세션 정보가 복제되어있음.
- 한번만 요청 했는데 다른 서버로 복제하는 추가 요청이 발생
- 복제하는 만큼 자원 소모 증가

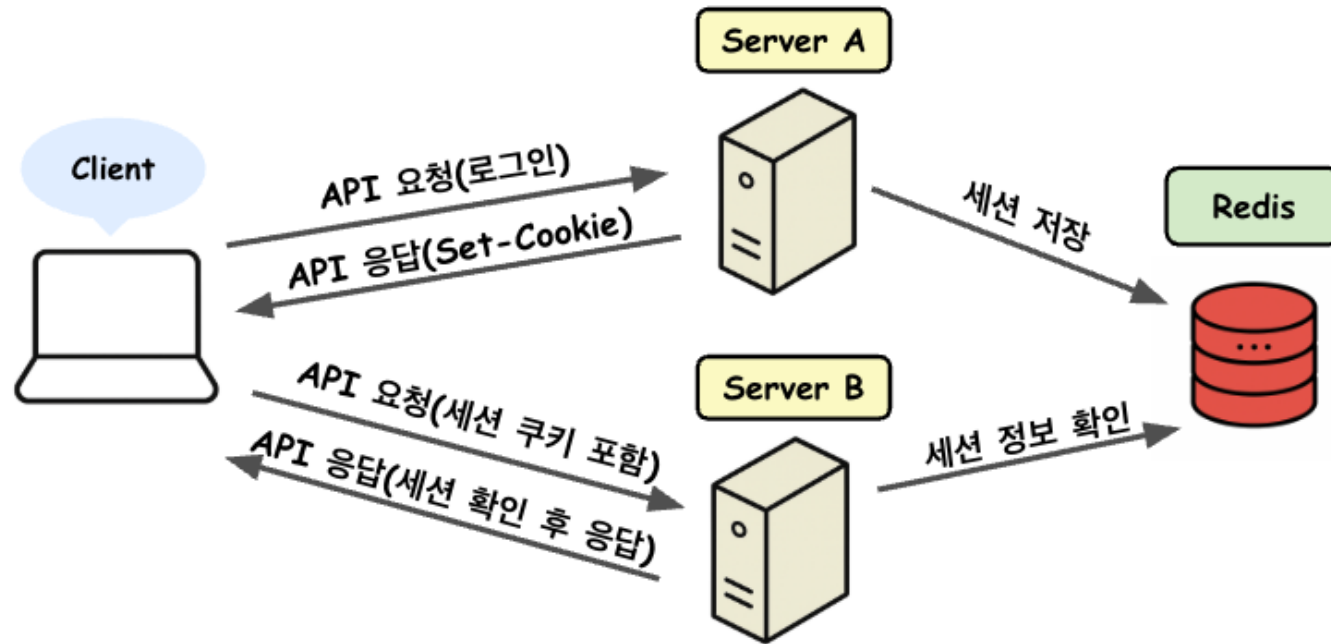


6. session & JWT(AT, RT) 저장 위치

6.4. Redis(외부 저장소) - 권장

- 세션 정보를 한 곳에만 저장해도 된다 (동기화 불필요)
- 어느 서버로 요청이 분산되어도 세션 정보에 접근할 수 있다.
- 세션 만료 기간도 redis에서 쉽게 설정 가능하다.
- Redis 서버에 대한 추가적인 관리가 필요하다.

Multi Server



7. JWT(AT, RT) 저장 위치

AT
이메일: 123
발급일: yy.mm.dd
유효기간:3분

저장위치	새로그침 후	장점	단점	권장도	비고
메모리 (JS 변수 / SPA 상태)	사라짐(새로그침 시)	XSS 리스크 최소화, SPA에 적합	새로그침 시 재발급 필요 (복구로직 필요)	권장 (SPA)	RT + 자동재발급 조합 권장
HttpOnly 쿠키	유지	자동 전송, XSS로부터 강함	CSRF 대비 필요 (SameSite/CSRF 토큰)	권장(옵션)	AT를 쿠키에 둘 때 CSRF 설계 필수
localStorage	유지	사용·구현 쉬움	XSS에 매우 취약 – 비권장	비권장	RT 절대 금지
sessionStorage	탭 닫으면 삭제	탭 단위 보관 (세션성)	XSS 취약 – 비권장	비권장	SPA에서도 새 탭엔 안씀

7. JWT(AT, RT) 저장 위치

Refresh token
Rt_id: 3
유효기간:3일

저장위치	새로고침 후	장점	단점	권장도	비고
HttpOnly 쿠키 (권장)	유지	XSS로부터 강함, 자동 전송으로 편리	CSRF 대비 필요, 쿠키 설정 신중	강력 권장 (웹)	Path=/auth/refresh, Secure, SameSite 권장
서버 저장소 (DB/Redis) — 서버 측 RT 관리	N/A (서버에 보관)	서버가 직접 추적·무효화 가능(보안 ↑)	/refresh 호출 시 서버 조회 필요(부하)	권장(추적 필요시)	RT 원문 대신 jti 해시 + status 저장 권장
localStorage / sessionStorage	유지 / 탭 닫으면 삭제	구현 쉬움	XSS에 취약 — 절대 권장X	금지권고	RT는 절대 저장 금지 권장
메모리 (JS 변수)	사라짐	XSS 노출 시간 줄임	브라우저 새로고침/탭 닫힘 시 유실 → UX 불편	보통 비권장	RT는 장기 보관용이므로 부적합
모바일 보안 저장소 (Keychain/Keystore)	유지	안전한 장기 저장소 — 권장	플랫폼별 구현 필요	권장(모바일)	RT 보관 시 우선 고려

QnA